Embedded Systems

Input/Output

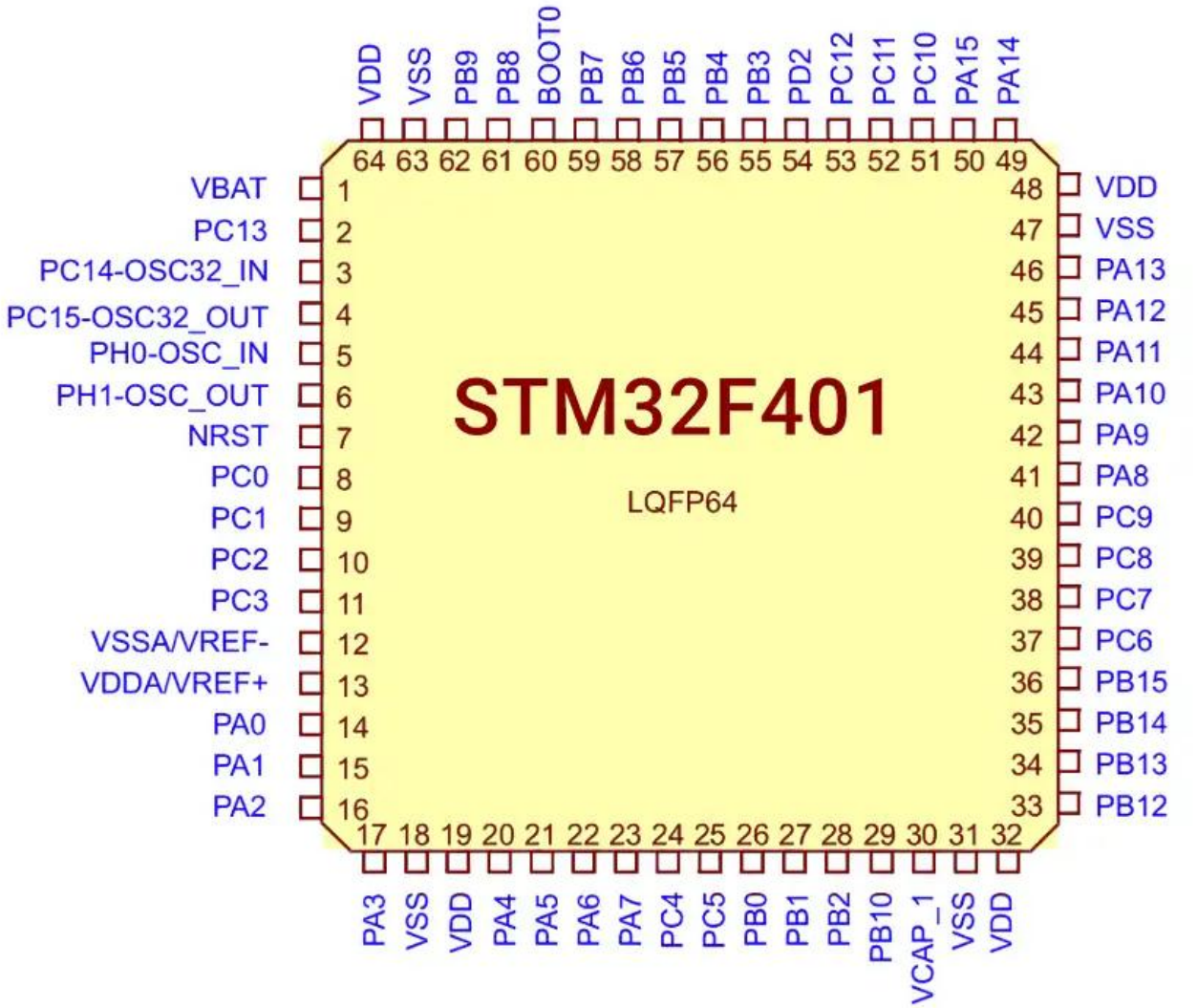
Topics

- Input / Output on ARM Cortex-M (AC6 assembly)
- GPIO,
- GPIO_AFR,
- UART/ USART
- SPI,
- LEDs,
- 7-segment

Memory-mapped I/O & Peripheral Model

- Peripherals are mapped into address space; registers accessed with LDR/STR.
- Typical steps to use a peripheral:
  - Enable peripheral clock (RCC)
  - Configure GPIO pins (MODER / AFR)
  - Configure peripheral registers (CR1/CR2/BRR/…)
  - Enable peripheral
  - Use poll or interrupt to transfer data
- Example:
  - LDR R0, =GPIOA_BASE
  - Use STR/LDR [R0, offset] to access them.

STM32F401 Pinout Diagram



Example: Base Addresses (STM32F4 family)

- RCC_BASE = 0x40023800
- GPIOA_BASE = 0x40020000
- GPIOB_BASE = 0x40020400
- USART2_BASE= 0x40004400
- SPI1_BASE = 0x40013000
- I2C1_BASE = 0x40005400

- Note: these addresses are examples for STM32F4. Always check your reference manual for your MCU.

GPIO Registers

- **MODER (0x00)** — mode: 00=input, 01=output, 10=AF, 11=analog (two bits per pin)
- **OTYPER (0x04)** — push-pull / open-drain
- **OSPEEDR (0x08)** — speed
- **PUPDR (0x0C)** — pull-up/pull-down
- **IDR (0x10)** — input data register (read pins)
- **ODR (0x14)** — output data register (read/write)
- **BSRR (0x18)** — bit set/reset (atomic set/reset)
- **AFRL (0x20), AFRH (0x24)** — alternate function select

OTYPER

- The **OTYPER** (Output Type Register) is a 32 bit to configure the **output type** of each pin when it's set as a digital output.
- Each bit corresponding to a single GPIO pin (only 16 bits are used).
- **Push-Pull Mode** (OTYPER bit = 0, the default output mode).
- **Open-Drain Mode** (OTYPER bit = 1)

OSPEEDR

- **OSPEEDR** (Output Speed Register) controls the **slew rate** of each pin when configured as an output.
- Slew describes how quickly a voltage level can transition from one state to another
- OSPEEDR uses **2 bits per pin**.
  - 00: Low speed
  - 01: Medium speed
  - 10: High speed
  - 11: Very high speed

PUPDR

- The **PUPDR** (Pull-up/Pull-down Register) is used for defining a default state for a pin that is not actively driven by an external device.
- PUPDR is used primarily for input pins, and determines the pin's state when nothing is actively driving it.
- This prevents the pin from floating and picking up electrical noise.
  - 00 No pull-up/pull-down
  - 01 Pull-up
  - 10 Pull-down
  - 11 Reserved

IDR (0x10) / ODR (0x14)

- **IDR** (Input Data Register) : Read-only
- When a pin is configured as an input, reading its corresponding bit in the IDR tells you whether the pin's voltage is currently high (1) or low (0)

- **ODR** (Output Data Register) : Read/Write
- When a pin is configured as a digital output, writing a 1 or 0 to its corresponding bit in the ODR sets the pin's output voltage to high or low, respectively.

BSRR (0x18)

- The **BSRR** (Bit Set/Reset Register) allows to set or reset individual pins with a **single atomic write operation**.

- **The Bit Set half** (Bits 0-15): Writing a 1 to any of these bits will set the corresponding pin to a high state.
- **The Bit Reset half** (Bits 16-31): Writing a 1 to any of these bits will reset the corresponding pin to a low state.
- Writing a 0 has no effect.

AFRL (0x20), AFRH (0x24)

- **AFR** (Alternate Function Register) is used to select the alternate function for each GPIO pin.
- AFRL (Alternate Function Register Low): Configures the alternate function for the **lower 8 pins** of a GPIO port
- AFRH (Alternate Function Register High): Configures the alternate function for the **upper 8 pins** of a GPIO port.
- Each group of **4 bits** configures one pin.
- Note: The pin's mode should be set to Alternate Function Mode using the MODER register

GPIO: Common Sequence

- Enable RCC AHB1 clock for GPIO port.
- Set MODER for each pin (input/output/AF).
- If AF: set AFRL/AFRH to AF number.
- Configure OTYPER/PUPDR/OSPEEDR if needed.
- Read IDR or write ODR/BSRR.

Enabling Clock

- RCC_AHB1ENR register is the **Reset and Clock Control (RCC) peripheral** of STM32 microcontrollers (and other ARM Cortex-M MCUs from STMicroelectronics).
- Each bit in this register controls the clock for one peripheral.
- GPIOA clock enable = bit 0
- GPIOB clock enable = bit 1
- GPIOC clock enable = bit 2
- .....
- If the clock is not enabled for a peripheral, we cannot access its registers.

Advanced Peripheral Bus vs Advanced High-performance Bus

- RCC (Reset and Clock Control) provides the clock to **APB1** (42 MHz) and **APB2** (84 MHz) buses.
- USART1 and USART6 are on the APB2 bus (higher speed).
- USART2 and UART4 are on the APB1 bus (lower speed).
- Each USART/UART has its own enable bit in RCC_APB1ENR or RCC_APB2ENR register.
- **AHB** (Advanced High-performance Bus) is a high-speed bus designed for connecting high-performance components for fast data transfer and low latency

Example: Blink LED on PA5 (AC6)

```
; ---------- constants ----------
GPIOA_BASE  EQU 0x40020000
RCC_AHB1ENR EQU 0x40023830 ; GPIO ports use AHB Bus
; ---------- init ----------
    LDR     R0, =RCC_AHB1ENR
    LDR     R1, [R0]
    ORR     R1, R1, #(1<<0)      ; enable GPIOA clock
    STR     R1, [R0]

    LDR     R0, =GPIOA_BASE
    LDR     R1, [R0, #0x00]      ; MODER
    BIC     R1, R1, #(3<<(5*2))  ; clear MODER for PA5 (bits 10:11)
    ORR     R1, R1, #(1<<(5*2))  ; set PA5 as general output (01)
    STR     R1, [R0, #0x00]
```

Example: Blink LED on PA5 (AC6)

```
loop:
    LDR    R0, =GPIOA_BASE
    MOV    R1, #(1<<5)
    STR    R1, [R0, #0x18]      ; BSRR set PA5
    BL     delay
    MOV    R1, #(1<<(5+16))
    STR    R1, [R0, #0x18]      ; BSRR reset PA5
    BL     delay
    B      loop
 ; delay: simple busy-loop
delay:
    MOV    R2, =100000
Count:
    SUBS   R2, R2, #1
    BNE    Count
    BX     LR
```

Using GPIO Alternate Function (AFR)

- Alternate functions map pins to peripherals (UART, SPI, I2C, TIM, etc).
- AFRL handles pins 0–7 (4 bits per pin),
- AFRH handles pins 8–15.

- **Steps**:
- Set MODER to 10 (Alternate Function),
- Set AFRL/H[n+3:n]=AFn.

USART Structure

- **Registers (USARTx):**

- **SR (status)** : includes flags
  - TXE (Tx empty), bit 7
  - TC (transmission complete), bit 6
  - RXNE (Rx not empty) bit 5
- **DR (data) :** hold the data being transmitted or received
- **BRR (baud rate)** : set baud using peripheral clock
- **CR1** : CR1 register contains bits to configure the following:
  - **UE (USART Enable)**: A bit to turn the USART peripheral on or off.
  - **M (Word Length)**: A bit to select the number of data bits per frame (e.g., 8 or 9 bits).
  - **PCE (Parity Control Enable)**: A bit to enable or disable parity generation and checking.
  - **TE (Transmitter Enable)**: A bit to enable the transmitter section.
  - **RE (Receiver Enable)**: A bit to enable the receiver section.
- **CR2/CR3** — advanced options (stop bits, flow control)

USART Structure

- **Steps to use USART:**
- Enable USART clock (RCC)
- Configure GPIO pins to AF (TX/RX)
- Set BRR
- Configure CR1 (enable TE, RE)
- Enable USART (UE)

Ex.: Configure PA2/PA3 for USART2 (TX/RX)

```
;Assume PA2 => AF7, PA3 => AF7.
GPIOA_BASE  EQU 0x40020000
RCC_AHB1ENR EQU 0x40023830
    LDR    R0, =RCC_AHB1ENR
    LDR    R1, [R0]
    ORR    R1, R1, #(1<<0)     ; enable GPIOA clock
    STR    R1, [R0]
    LDR    R0, =GPIOA_BASE
    LDR    R1, [R0, #0x00]          ; MODER
    BIC    R1, R1, #(3<<(2*2))     ; clear PA2 MODER
    BIC    R1, R1, #(3<<(3*2))     ; clear PA3 MODER
    ORR    R1, R1, #(2<<(2*2))     ; PA2 AF (10)
    ORR    R1, R1, #(2<<(3*2))     ; PA3 AF
    STR    R1, [R0, #0x00]
    LDR    R1, [R0, #0x20]           ; AFRL
    BIC    R1, R1, #(0xF<<(4*2))    ; clear AF for PA2
    BIC    R1, R1, #(0xF<<(4*3))    ; clear AF for PA3
    ORR    R1, R1, #(7<<(4*2))       ; AF7 for PA2
    ORR    R1, R1, #(7<<(4*3))       ; AF7 for PA3
    STR    R1, [R0, #0x20]
```

Example: Send a Byte over USART2 (AC6)

```
  USART2_BASE EQU 0x40004400
    RCC_APB2ENR, EQU 0x40023844
    LDR R0, =RCC_APB2ENR
    LDR R1, [R0]
    ORR R1, R1, #(1 << 12)
    STR R1, [R0]
    LDR     R0, =USART2_BASE
    MOV     R1, #0x0000          ; clear
    STR     R1, [R0, #0x0C]      ; CR1 = 0
    MOV     R1, #0x0683          ; BRR for 9600@16MHz (example)
    STR     R1, [R0, #0x08]      ; USART_BRR
    MOV     R1, #(1<<13)|(1<<3)|(1<<2) ; UE(13)=1, TE(3)=1, RE(2)=1
    STR     R1, [R0, #0x0C]      ; enable USART, TX & RX

  send_byte:
    LDR     R1, [R0, #0x00]      ; SR
    TST     R1, #(1<<7)          ; check TXE (bit7)
    BEQ     send_byte
    MOV     R2, #'A'
    STR     R2, [R0, #0x04]      ; write to DR
```

USART Receive (Polling)

- To receive a byte from USART using polling we have to check bit 5 of the status register.
- Polling:
- ```
  LDR R1, [R0, #0x00]    ; SR
  TST R1, #(1<<5)        ; RXNE (bit5)
  BEQ Polling
  LDR R2, [R0, #0x04]    ; DR -> received char
  ```
  ```

SPI Fundamentals

- Key registers (SPIx):
- **CR1** (Control Register 1):
- Bits are:
  - SPE (SPI Enable): set 1 to enable
  - MSTR (Master Selection): 1 indicates peripheral operates as master
  - BR (Baud Rate Control): Determines data rate using peripheral clock
  - CPOL (Clock Polarity): sets the idle state of the clock signal. '0' means the clock is low when not transmitting
  - CPHA (Clock Phase): determines when the data is sampled relative to the clock's transition. '0' means data is sampled on the first edge of the clock cycle.

SPI Fundamentals

- Key registers (SPIx):
- **CR2** (Control Register 2).
- Bits used:
  - RXDMAEN (Rx Buffer DMA Enable)
  - TXDMAEN (Tx Buffer DMA Enable)
  - SSOE (SS Output Enable): SS pin is automatically driven **low** when the SPI communication **starts** and driven **high** when it's **finished**.
  - NSS (Negative Slave Select): a device is selected when its NSS pin is driven **low**.

SPI Fundamentals

- Key registers (SPIx):
- **SR** (Status Register):
- Bits are:
  - RXNE (Receive Buffer Not Empty)
  - TXE (Transmit Buffer Empty)
  - BSY (Busy Flag): indicates that a communication transaction is currently in progress.
- **DR** : data register

SPI Usage

- Sequence:
- Enable SPI clock
- Configure GPIO pins to AF (SCK/MISO/MOSI), and NSS (if used)
- Set CR1 for master, CPOL/CPHA, data size
- Enable SPI
- Transmit by writing to DR; check TXE and BSY

Example: SPI Master Transmit One Byte

```
SPI1_BASE   EQU 0x40013000

    LDR     R0, =SPI1_BASE
    LDR     R1, [R0, #0x00]      ; read CR1
    ORR     R1, R1, #(1<<2)      ; MSTR bit (master)
    BIC     R1, R1, #(1<<1)      ; CPOL=0
    BIC     R1, R1, #(1<<0)      ; CPHA=0
    ORR     R1, R1, #(1<<6)      ; SPE = 1 (enable) - often set after config
    STR     R1, [R0, #0x00]      ; CR1

    ; Transmit byte
    MOV     R2, #0x5A
wait_tx:
    LDR     R1, [R0, #0x08]      ; SR
    TST     R1, #(1<<1)          ; TXE?
    BEQ     wait_tx
    STR     R2, [R0, #0x0C]      ; write DR
; wait for completion
wait_bsy:
    LDR     R1, [R0, #0x08]
    TST     R1, #(1<<7)          ; BSY?
    BNE     wait_bsy
```

Reading a GPIO sensor (push button on PA0)

- Configure PA0 as input (MODER = 00)
- Configure PUPDR to enable pull-down or pull-up
- Poll IDR
- Example (read & branch):

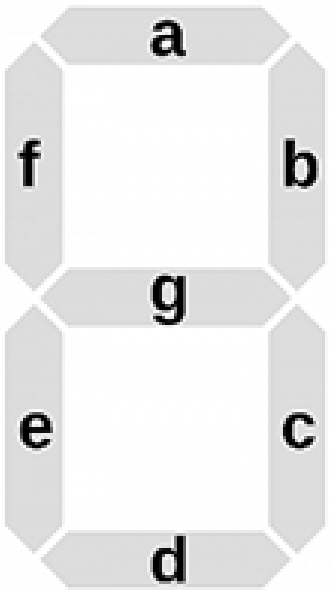
```
    LDR     R0, =GPIOA_BASE
    LDR     R1, [R0, #0x10]       ; IDR
    TST     R1, #(1<<0)           ; check PA0
    BEQ     not_pressed
pressed:
    ; handle pressed
    B       continue
not_pressed:
    ; handle not pressed
continue:
```

Driving an LED

- Digital: write ODR or BSRR (atomic).
- Example: write ODR or set via BSRR.
- Atomic set/reset with BSRR:
- MOV R1, #(1<<5)
- STR R1, [R0, #0x18] ; set PA5
- MOV R1, #(1<<(5+16))
- STR R1, [R0, #0x18] ; reset PA5

7-Segment Display: Wiring & Patterns

- Common cathode: segments high to light. Common anode: segments low to light.
- Segments mapping example (bits -> segments):
- bit7 DP
- bit6 G
- bit5 F
- bit4 E
- bit3 D
- bit2 C
- bit1 B
- bit0 A
- Digit patterns (common cathode) (A..G,DP):
- 0: 0b00111111 ; segments A B C D E F
- 1: 0b00000110 ; B C
- 2: 0b01011011 ; A B D E G
- 3: 0b01001111 ; A B C D G
- 4: 0b01100110 ; B C F G
- 5: 0b01101101 ; A C D F G
- 6: 0b01111101 ; A C D E F G
- 7: 0b00000111 ; A B C
- 8: 0b01111111 ; all
- 9: 0b01101111 ; A B C D F G



Ex: Output Digit to 7-segment (Single Digit)

- Assume GPIOB pins [0..7] connected to segments A..DP.
- GPIOB_BASE EQU 0x40020400
-
- ; patterns in memory or constants
- PATTERN_5   EQU 0x6D   ; 0b01101101
-
-
- LDR   R0, =GPIOB_BASE
- MOV   R1, #PATTERN_5
- STR   R1, [R0, #0x14]   ; write ODR

Multiplexing Multiple 7-segment Digits

- Sample Practice: Use per-digit enable pins (common anode/cathode) + segment bus shared.
- Cycle digits quickly (multiplex) to show multiple digits.
- Timing: refresh each digit ~1–5 ms.
- Pseudo:
- for digit in digits:
- set segments for value[digit]
- enable digit (activate its common pin)
- delay (1-3 ms)
- disable digit

Common pitfalls & best practices

- Always enable the peripheral clock before touching registers.
- Set pin AF before enabling peripheral.
- Use BSRR for atomic bit set/reset (avoid read-modify-write races).
- Check and clear status flags as required (reading SR/DR pattern).
- For I²C & SPI: check datasheet for timing & required delays.
- When using interrupts: keep ISRs short; use buffers for heavy work.

Questions?